# 8-Bit Linear Phase FIR Filter (Part 3)

Devansh Modi (202304009)

**Abstract—In this assignment, we implemented an 8-bit linear-phase FIR low-pass filter in MATLAB and structural Verilog. Starting from the specifications derived in Part 1, the filter is implemented with a sampling frequency of 4 kHz using Q7 fixed-point representation for both coefficients and input samples. A linear-phase architecture is chosen to exploit coefficient symmetry, which reduces multiplication and addition requirements, thereby lowering hardware cost and power consumption. MATLAB/Simulink modelling verifies the functional correctness of the filter for various input test cases, including mixed-frequency signals. The Verilog implementation uses signed carry-save multipliers and CLA adders. Overall, the design achieves accurate low-pass filtering, stable fixed-point behaviour, and an efficient hardware structure suitable for DSP applications.**

## I. Introduction

In this part 2 of the assignment, we are going to implement the FIR filter designed in part 1. We are going to implement it in MATLAB and Verilog.

The cutoff frequency ($F_C$) of the filter is determined by the sampling frequency ($F_S$) of the input signal. To get the desired cutoff frequency of 1 KHz, the sampling frequency can be given by,

$$F_S = 2 \cdot \left(\frac{F_C}{F_N}\right) \quad ; F_N = \text{Normalized cutoff frequency} = 0.5$$

By substituting the values of $F_C$ and $F_N$ we get the sampling frequency as 4 KHz. We will generate the input signal with this sampling frequency of 4 kHz in MATLAB. The coefficients and input samples were quantized by using the fi object in MATLAB. The 'fi' object in MATLAB uses signed 2's complement fixed-point representation (Q7) for storing values.

## II. Filter Structure

The FIR filter is characterized by the transfer function, H(z), given by,

$$\boxed{H(z) = \sum_{n=0}^{14} h(n)\, z^{-n}}$$

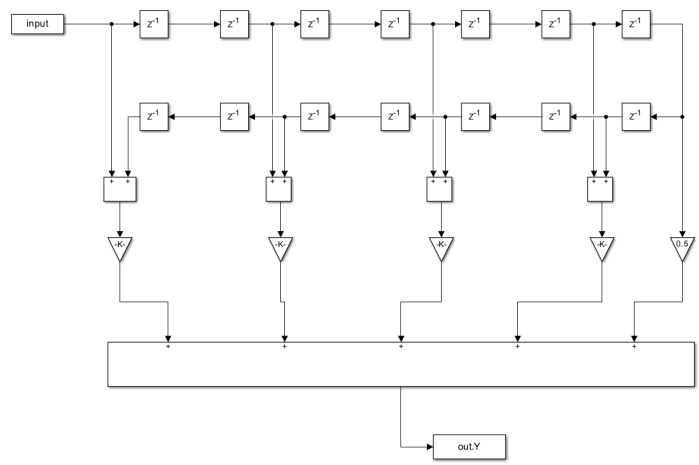
There are mainly two types of realizations: Transversal & Linear Phase. For our filter design, we have used a Linear Phase structure, as we get the advantage of symmetrical filter coefficients. It reduces the computational complexity of filter implementation. Other advantages of using Linear Phase structure are that it takes less area and consumes less power due to a reduced number of multiplication and addition operations.

For the designed filter structure, the computation of each output sample, y(n), requires

- 5 memory locations (ROM) to store 5 coefficients
- 14 RAM locations to store delayed samples of input
- 5 Multiplication Operation
- 8 Addition Operation

## II. MATLAB Implementation

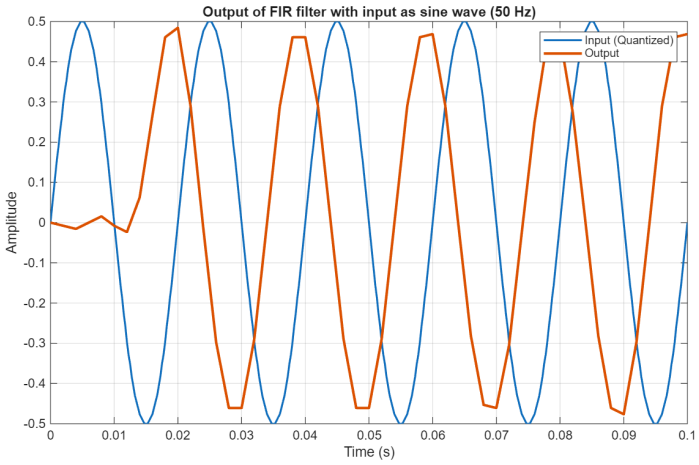
In MATLAB, we used Simulink to model our filter. The primary goal of implementing it in MATLAB was to verify the functionality of the desired filter. As a result, the adder and multiplication modules were implemented at a high level of abstraction, without incorporating any gate delays.



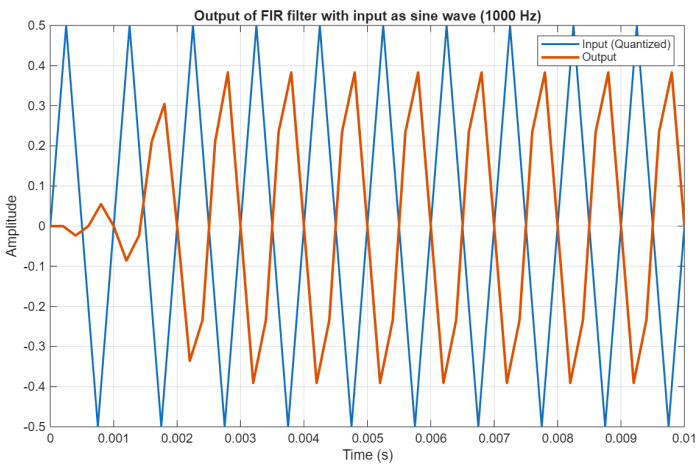

(Fig. 1. Simulink Model of FIR filter)

In MATLAB the Multiplication block has an in-built option called Integer rounding Mode which can be set to zero. This simply means discarding LSB bits without rounding.

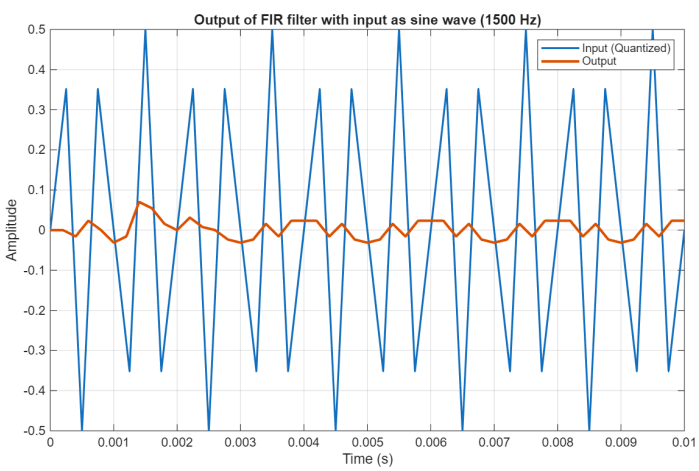
To check the functionality of the filter different combinations of sine waves were given as the input to the filter. Figure 2 to 7 shows the output of the filter to different sine wave signals given as input.




(Fig.2. Output of the FIR filter with input as sine wave(50Hz)

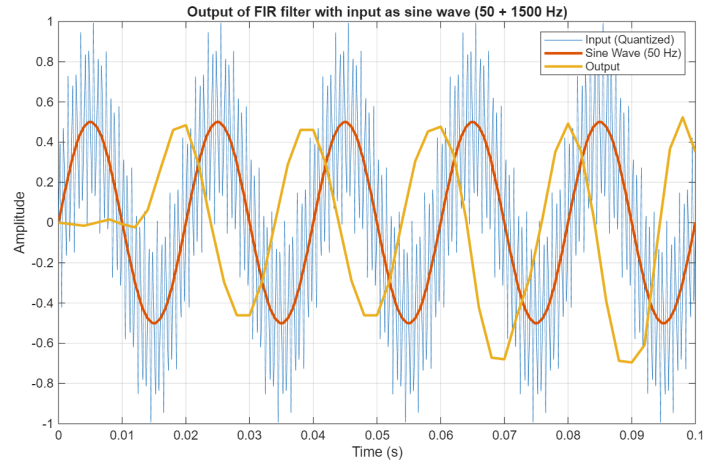



(Fig.3.Output of FIR Filter with input as sine wave(1000Hz)

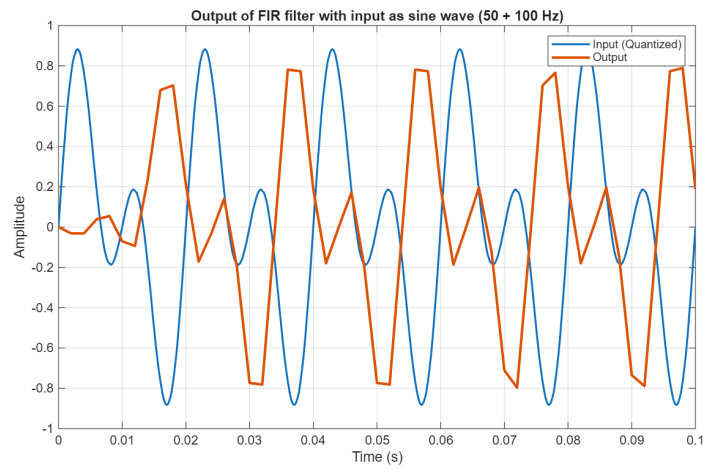



(Fig.4.Output of FIR Filter with input as sine wave(1500 Hz))

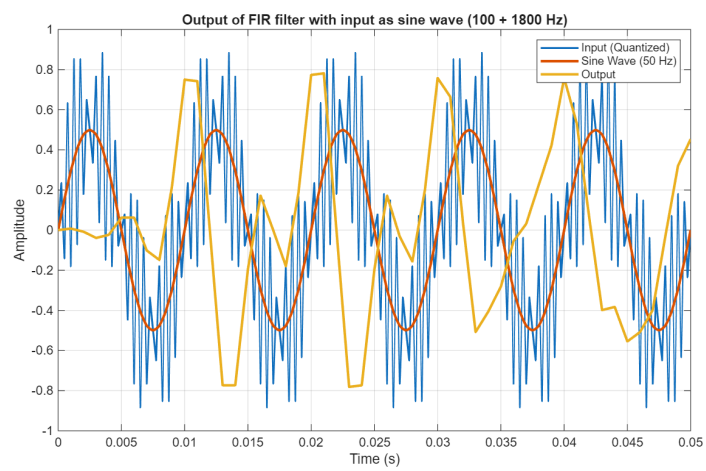
We mixed two sine waves: one with a frequency below the cutoff frequency and the other with a frequency above the cutoff frequency, and then we applied this to the filter.




(Fig.5.Output of FIR Filter with input sine wave(50Hz+1500Hz)




(Fig.6.Output of FIR Filter with input sine wave(50Hz+100Hz)




(Fig.7.Output of FIR Filter with input sine wave (100+1800Hz))

Figure 7 shows a particular case when arithmetic overflow occurred. We can observe that the low frequency component signal was not reconstructed accurately because of overflow.

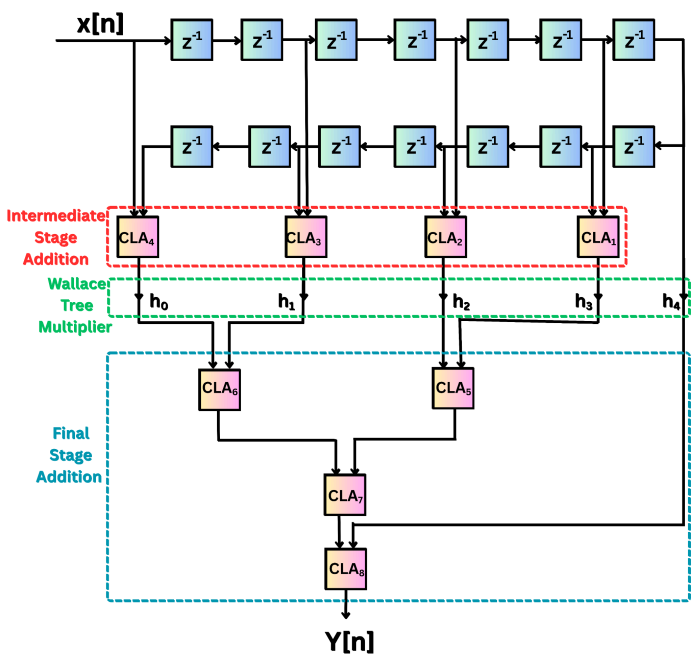
## III. Verilog Implementation

For this assignment, we are going to use typical 0.18 µm CMOS delays as a reference. For example, FO4 loads at 180 nm give an inverter delay ≈60 ps and a 2-input NAND ≈72 ps . Using these as a baseline, we estimate the following propagation delays at 1.8 V, for a "standard" load (roughly FO4):

- **2-input NAND** ≈ 72 ps (0.072 ns)
- **2-input AND** ≈ 72 ps + 60 ps ≈ 132 ps (0.13 ns)
- **3-input AND** ≈ 170 ps (0.17 ns)
- **4-input AND** ≈ 210 ps (0.21 ns)
- **2-input XOR** ≈ 240 ps (0.24 ns)
- **2-input OR** ≈ 180ps (0.18 ns)
- **3-input OR** ≈ 230 ps (0.23 ns)
- **4-input OR** ≈ 290 ps (0.29 ns)

All values above are approximate and assume typical (FO4) loading at 1.8 V. They are derived from known 180 nm CMOS characterizations (e.g. FO4 INV=60 ps, NAND2=72 ps ) and logical effort scaling. Exact rising vs. falling delays are device-dependent; the numbers given are rough averages.

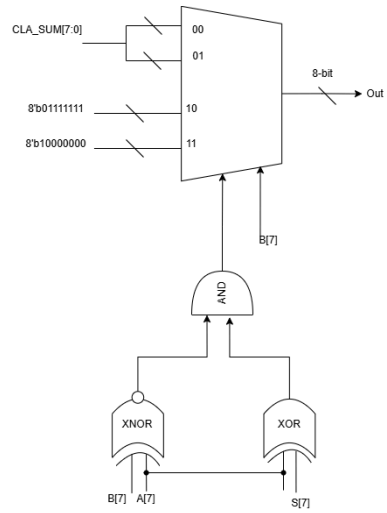
Figure 8 shows the structure of the FIR filter that we are going to implement in structural verilog. The symbol $z^{-1}$ represents a delay of one sample or unit of time. These delay elements will be implemented by shift registers.




(Fig. 8. Schematic representation of the FIR filter)

The intermediate addition operation is done using a Carry Look Ahead (CLA) adder, as it is a fast adder architecture as compared to other adders. The final addition of 5 products is done in three stages using CLA. <u>After doing exhaustive testing, the worst-case delay of CLA was found to be 2.37 ns.</u>

If any overflow occurs in CLA, we will saturate the value. Let us say input to the CLA is A[7:0], and B[7:0], and the output is S[7:0]. Positive overflow will occur when (A[7] = B[7]) & (S[7]!= 0). Similarly, a negative overflow will occur when (A[7] = B[7]) & (S[7]!= 1). Detecting the overflows and saturating the CLA output can be done by implementing the logic given in figure 9.




(Fig. 9. Logic for Overflow Detection and Saturation)

To perform signed multiplication, we have used a signed carry save multiplier because of its simple array structure and easy implementation. <u>By doing exhaustive testing on the multiplier, its worst-case delay was found to be 4.558 ns.</u>

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | $y_5$ | $y_4$ | $y_3$ | $y_2$ | $y_1$ | $y_0$ |
| | | | | | | $x_5$ | $x_4$ | $x_3$ | $x_2$ | $x_1$ | $x_0$ |
| | | | | | 1 | $\overline{x_5y_0}$ | $x_0y_4$ | $x_0y_3$ | $x_0y_2$ | $x_0y_1$ | $x_0y_0$ |
| | | | | | $\overline{x_5y_1}$ | $x_1y_4$ | $x_1y_3$ | $x_1y_2$ | $x_1y_1$ | $x_1y_0$ | |
| | | | | $\overline{x_5y_2}$ | $x_2y_4$ | $x_2y_3$ | $x_2y_2$ | $x_2y_1$ | $x_2y_0$ | | |
| | | | $\overline{x_5y_3}$ | $x_3y_4$ | $x_3y_3$ | $x_3y_2$ | $x_3y_1$ | $x_3y_0$ | | | |
| | | $\overline{x_5y_4}$ | $x_4y_4$ | $x_4y_3$ | $x_4y_2$ | $x_4y_1$ | $x_4y_0$ | | | | |
| 1 | $x_5y_5$ | $\overline{x_4y_5}$ | $\overline{x_3y_5}$ | $\overline{x_2y_5}$ | $\overline{x_1y_5}$ | $\overline{x_0y_5}$ | | | | |
| $p_{11}$ | $p_{10}$ | $p_9$ | $p_8$ | $p_7$ | $p_6$ | $p_5$ | $p_4$ | $p_3$ | $p_2$ | $p_1$ | |

(Fig.10.Simplified partial products for signed array multiplier)

Now that we have the worst-case delay of CLA and Multiplier, we can estimate the worst-case delay of the filter, i.e., the maximum time the filter takes to produce 1 output sample. The worst-case delay can be estimated as,

$$\mathbf{T_{FILTER} = (4*T_{CLA}) + T_{MULTIPLY}}$$

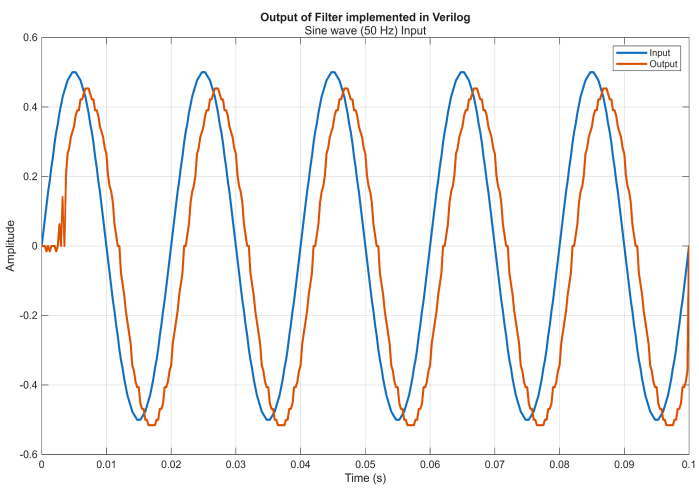
Substituting the values, we get the worst-case delay of the filter as **14.038 ns**. Therefore, the maximum possible frequency at which we can operate the filter is approximately **71 MHz**. But this high speed is of no use because the sampling frequency of the input signal is 4 kHz. We can optimize the filter circuit by trading off performance for more accurate results. For example, we can use double-length adders after multiplication and then truncate the final output.

**Verilog Code:**

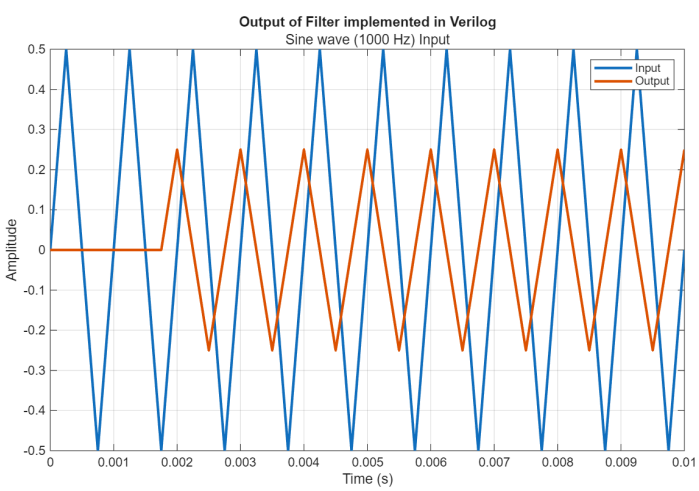
```
// Main module of FIR Filter

`timescale 1ns/1ps

module filter;

    reg signed [7:0] signal_in [0:999]; // Input
signal 8-bit wide in Q7 format

    reg signed [7:0] signal_out [0:999]; // Output
signal 8-bit wide in Q7 format

    reg signed [7:0] h [0:4]; // Registers to
store coefficients of filter

    reg signed [7:0] z [1:14]; // Shift register
to store delayed signal

    reg signed [7:0] x_in; // To store 1st sample
in a register

    reg clk; // Clock signal

    integer file1; // File handler

    reg [31:0] PC; // Program Counter (32-bit)

    integer i;


    // Calling adder module to calculate 4
intermediate Additions

    wire signed [7:0] w1 [0:3]; // Nets
representing intermediate addition results

    CLA8 ADD1(w1[0], x_in, z[14]);

    CLA8 ADD2(w1[1], z[2], z[12]);

    CLA8 ADD3(w1[2], z[4], z[10]);

    CLA8 ADD4(w1[3], z[6], z[8]);


    // Calling Multiplication module to calculate
5 intermediate Multiplications

    wire signed [7:0] w2 [0:4]; // Nets
representing intermediate multiplication results

    MUL8 M1(w2[0], w1[0], h[0]);

    MUL8 M2(w2[1], w1[1], h[1]);

    MUL8 M3(w2[2], w1[2], h[2]);

    MUL8 M4(w2[3], w1[3], h[3]);

    MUL8 M5(w2[4], z[7], h[4]);


    // Final stage addition

    wire signed [7:0] w3 [0:2];

    wire signed [7:0] filter_out; // Final output
of the filter

    CLA8 ADD5(w3[0], w2[0], w2[1]);

    CLA8 ADD6(w3[1], w2[2], w2[3]);

    CLA8 ADD7(w3[2], w3[0], w3[1]);

    CLA8 ADD8(filter_out, w3[2], w2[4]);


    initial begin

        clk = 1'b0;

        PC = 0;

       // Initialize the shift register to 0 value

        for (i = 1; i<15; i=i+1) begin

            z[i] = 8'b0;

        end

        // Reading input signal data from txt file

        $readmemb("data.txt", signal_in);

        // Loading Coefficients of filter

        $readmemb("coefficients.txt", h);

        // Opening a file to write the output data

        file1 = $fopen("Output.txt");

    end

    // Updating the shift register and input
samples on every clock edge

    always @(posedge clk) begin

        if(PC == 1000) begin

            $fclose(file1); // Close the file

            $display("Simulation finished
(processed %0d samples).", PC);

            $finish; // Finish the simulation

        end

        else begin

            // Store the computed output in memory
and write to the output file

            signal_out[PC] <= filter_out;

            $fdisplay(file1, "%b", filter_out);

            // Loading new samples on shift
registers

            x_in <= signal_in[PC];

            z[1] <= x_in;

            for (i = 2; i<=14; i=i+1) begin
```

```
            z[i] <= z[i-1];
        end
    PC <= PC + 1; // Update the program Counter
     end
  end
  // Clock signal Generation
  always #5 clk = ~clk;
endmodule
```

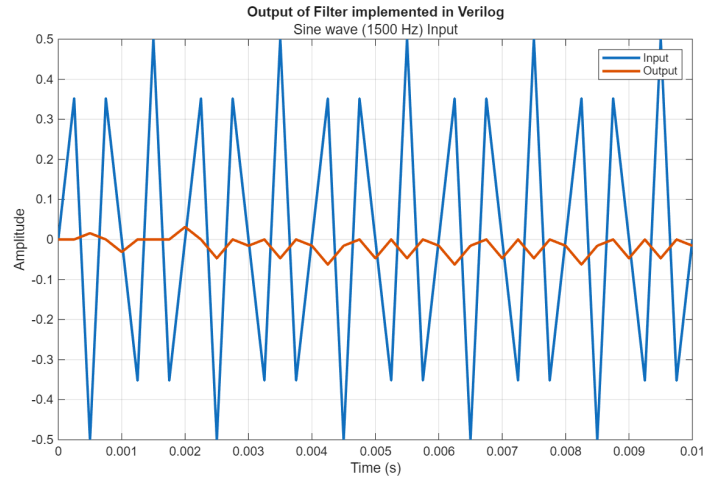
The input signal data was stored in a text file and was read during the execution of code. The output data was also written back to a text file. The signal was reconstructed from the text file in MATLAB. For testing the filter different combinations of sine waves were given as the input to the filter. The output of the filter is shown in figure 11 to 14.



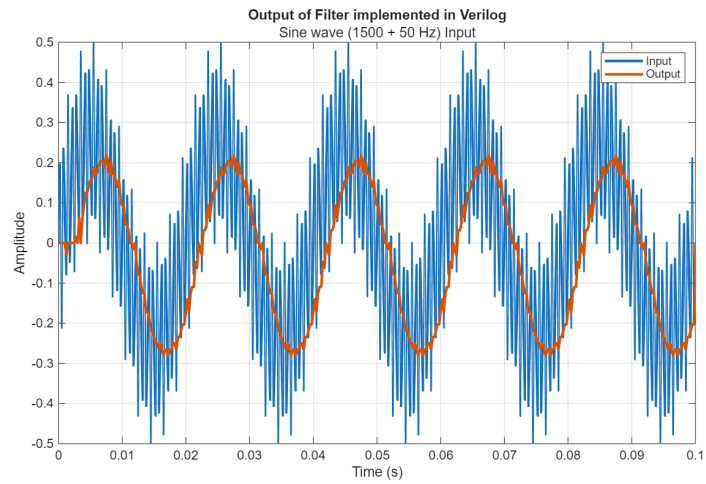

(Fig.11.Output of the filter to Sine wave (50Hz) input)




(Fig.12..Output of the filter to Sine wave (1000Hz) input)




(Fig.13..Output of the filter to Sine wave (1500Hz) input)




(Fig.14..Output of the filter to Sine wave (1500+50Hz) input)

## IV. Conclusion

In this work, an 8-bit linear-phase FIR low-pass filter was successfully implemented. MATLAB Simulink implementation confirmed correct low-pass behaviour for single and mixed frequency sine inputs, while the structural Verilog implementation—built using CLA adders and signed array multipliers—provided an accurate hardware model with a maximum operational frequency of ~71 MHz based on 0.18 µm CMOS delays. Same input signals were given to check verilog implementation.

## V. References

**[1]** Digital Signal Processing: A Practical Approach; Ifeachor, Jervis

**[2]** CMOS VLSI Design: A Circuits and Systems Perspective - Book by David Harris and Neil Weste.